

Generazione procedurale

Andrea Ranica - A.A. 2025/2026

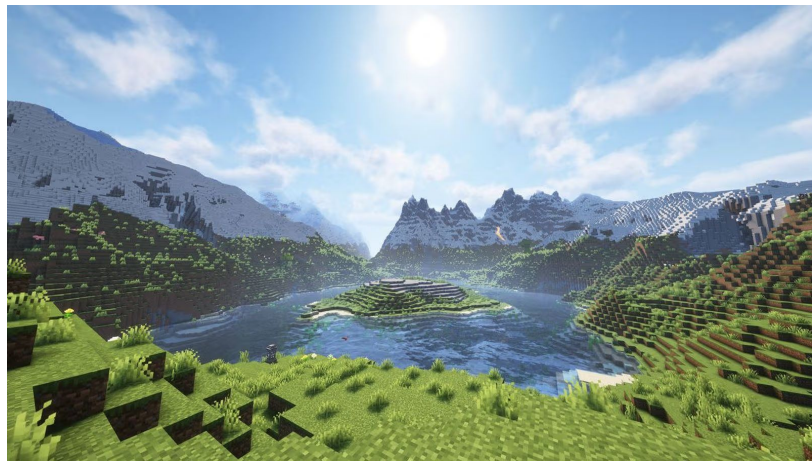
Generazione procedurale

Tecnica per creare dati e contenuti in modo automatico da un algoritmo.

Procedurale → dati generati con una procedura algoritmica (e non manualmente).



No Man's Sky, 2016



Minecraft, 2009

Generazione procedurale



Pro

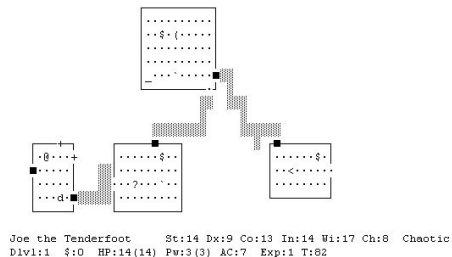
Risparmiare spazio: i dati non devono essere memorizzati ma vengono ottenuti *runtime* invocando una procedura

Gameplay variegato: variando i parametri degli algoritmi è possibile ottenere mondi diversi (No Man's Sky = $2^{64} \approx 18$ quintilioni di mondi)



Contro

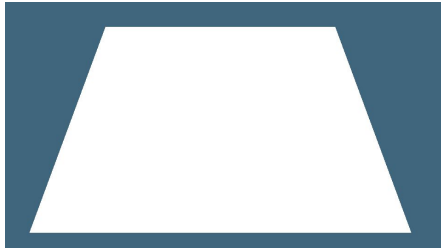
Risultati spesso ripetitivi: gli algoritmi utilizzano un numero fisso di parametri, quindi i risultati possono essere ripetitivi



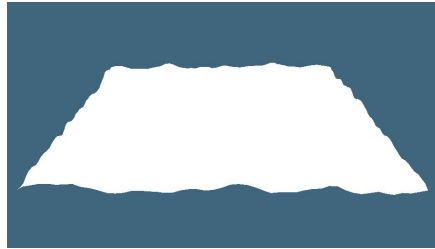
*Dungeon procedurali in
NetHack (1987)*

Generazione procedurale - Idea generale

1. Il mondo viene rappresentato con una mesh composta da facce triangolari



2. L'altezza di ogni vertice della mesh viene calcolata utilizzando una **funzione di rumore**

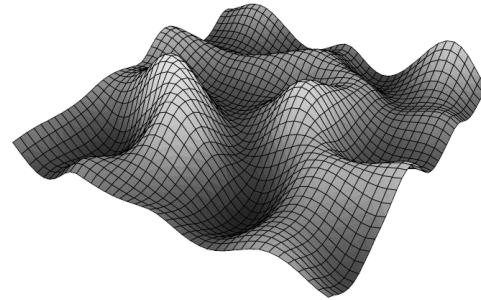
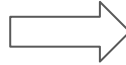
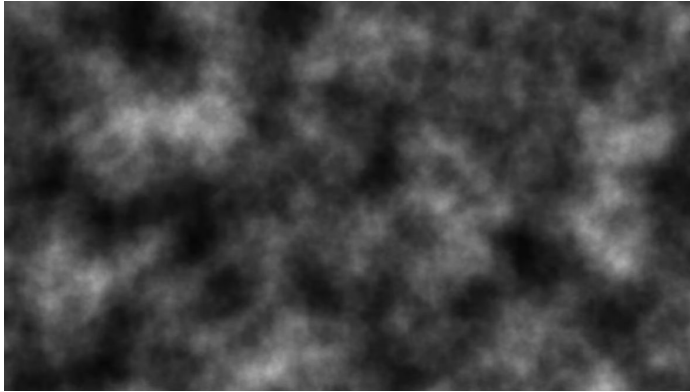


3. Le facce vengono colorate secondo la loro **pendenza**



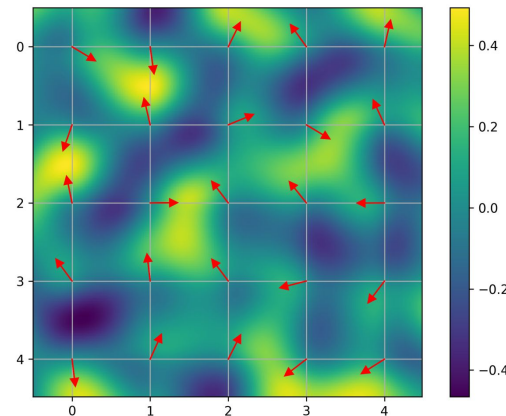
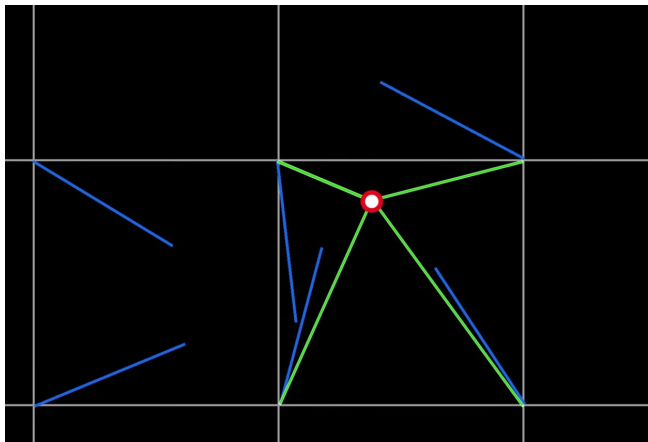
Perlin Noise - Definizione

Rumore a gradiente **pseudo-casuale** (*seed*): sequenza di numeri statisticamente casuali ottenuti da un processo deterministico → se il seed rimane costante anche il risultato è costante.



Perlin Noise - Funzionamento

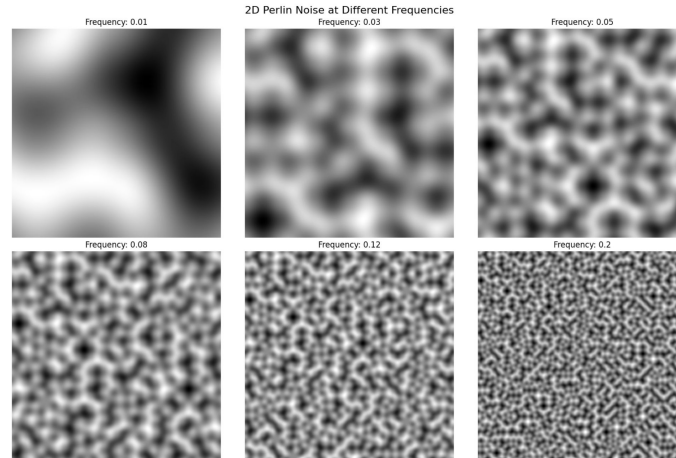
- Definizione di una griglia di celle con un vettore gradiente pseudocasuale associato a ogni vertice
- Dato un punto, si calcola il vettore distanza da ciascun vertice e si fa il prodotto scalare con il relativo gradiente
- Il valore finale è ottenuto eseguendo un'interpolazione (cubica) tra i prodotti scalari



Perlin Noise - Parametri

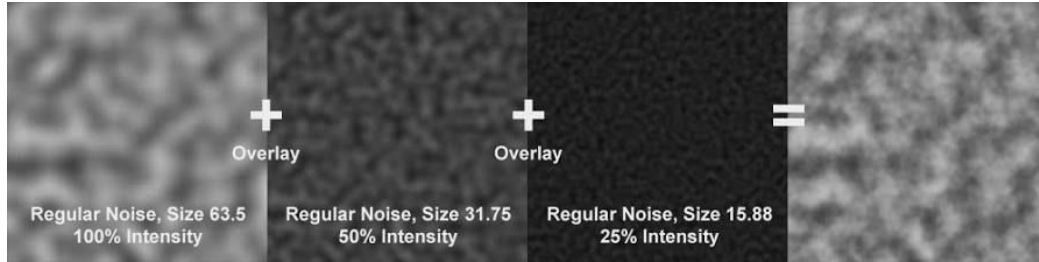
Il comportamento di Perlin Noise viene influenzato da due parametri:

- Frequenza: dimensione della griglia
- Ampiezza: intensità massima dei valori generati



Perlin Noise - Layering

Per ottenere un risultato più realistico e dettagliato vengono sommati più layer di rumore, in cui ogni layer è ottenuto aumentando la frequenza e diminuendo l'ampiezza del layer precedente.



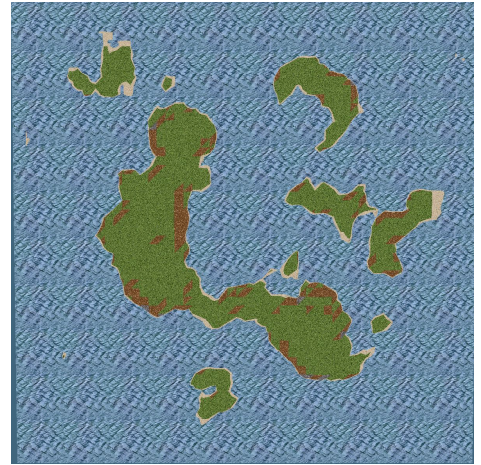
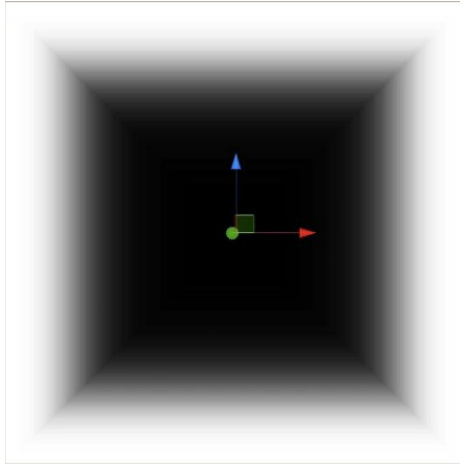
1 layer

6 layer

12 layer

Funzione di Falloff

Poiché Perlin Noise è continuo su $\mathbb{R}^2$ e non prevede i confini del mondo, è necessaria una funzione che riduca progressivamente l'altezza allontanandosi dal centro del mondo.

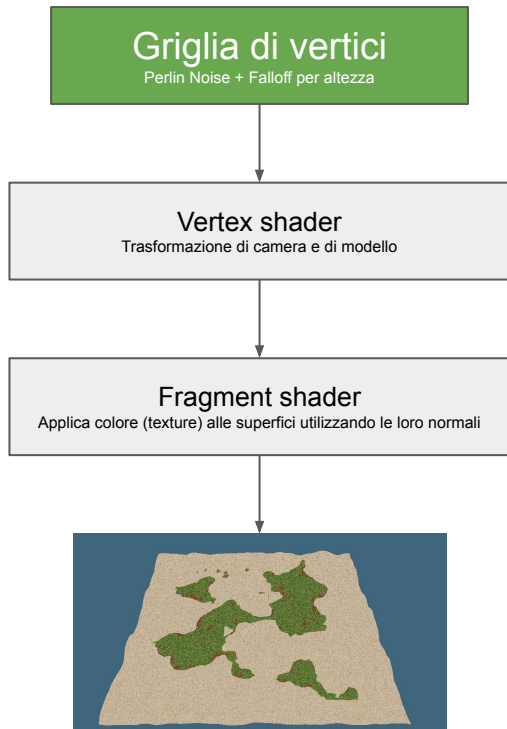


= 1

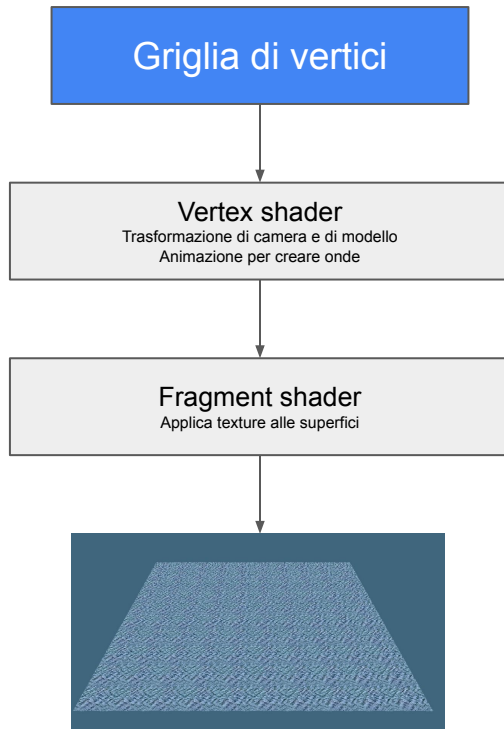
= 0

Pipeline semplificata

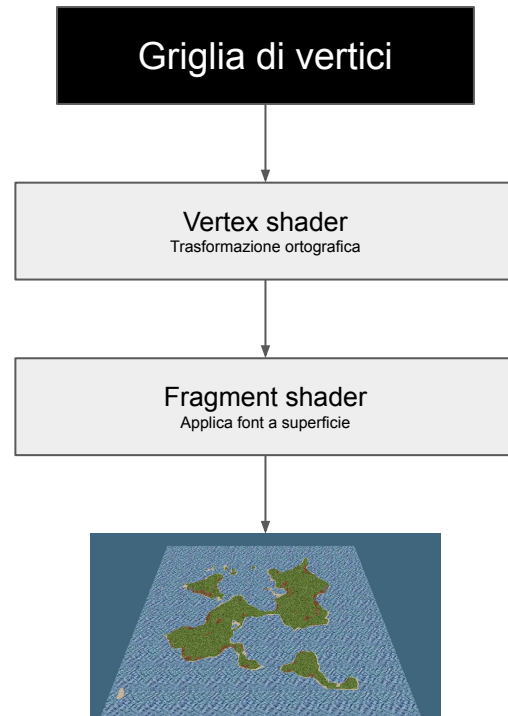
Terreno



Acqua



GUI



Rendering Terreno - Definizione della mesh

```
void World::regenerate_mesh(NoiseGenerator* noise_generator)
{
    std::vector<Vertex> vertices;
    for (int x = 0; x < height; x++)
    {
        for (int z = 0; z < width; z++)
        {
            int base = 6 * (x * width + z);

            glm::vec3 sw(x, get_point_height(x, z + 1, noise_generator), z + 1);
            glm::vec3 se(x + 1, get_point_height(x + 1, z + 1, noise_generator), z + 1);
            glm::vec3 ne(x + 1, get_point_height(x + 1, z, noise_generator), z);
            glm::vec3 nw(x, get_point_height(x, z, noise_generator), z);

            glm::vec3 first_face_normal = get_face_normal(sw, se, ne);
            glm::vec3 second_face_normal = get_face_normal(ne, nw, sw);

            // First triangle
            vertices.push_back(Vertex(sw, VERTEX_GRID, first_face_normal, glm::vec2(x, z + 1)));
            vertices.push_back(Vertex(se, VERTEX_GRID, first_face_normal, glm::vec2(x + 1, z + 1)));
            vertices.push_back(Vertex(ne, VERTEX_GRID, first_face_normal, glm::vec2(x + 1, z)));

            // Second triangle
            vertices.push_back(Vertex(ne, VERTEX_GRID, second_face_normal, glm::vec2(x + 1, z)));
            vertices.push_back(Vertex(nw, VERTEX_GRID, second_face_normal, glm::vec2(x, z)));
            vertices.push_back(Vertex(sw, VERTEX_GRID, second_face_normal, glm::vec2(x, z + 1)));
        }
    }

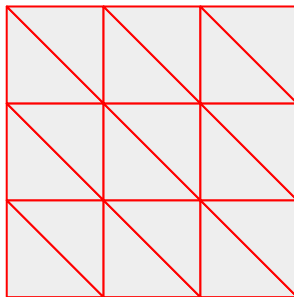
    glBindBuffer(GL_ARRAY_BUFFER, VBO);
    glBufferData(
        GL_ARRAY_BUFFER,
        vertices.size() * sizeof(Vertex),
        vertices.data(),
        GL_STATIC_DRAW);
    glBindBuffer(GL_ARRAY_BUFFER, 0);
}
```

```
float World::get_point_height (float x, float z, NoiseGenerator* noise_generator)
{
    float vertex_noise = noise_generator->get_noise(x, z);

    if (!enable_falloff)
    {
        return vertex_noise;
    }

    float falloff = FalloffGenerator::get_vertex_falloff(x, z, width, height);

    float vertex_height = vertex_noise - falloff;
    return vertex_height;
}
```



Rendering Terreno - Perlin Noise

```
vector2 NoiseGenerator::randomGradient(int ix, int iz) {  
    const unsigned w = 8 * sizeof(unsigned);  
    const unsigned s = w / 2;  
  
    unsigned a = ix + seed, b = iz + seed;  
    a *= 3284157443;  
    b ^= a << s | a >> w - s;  
    b *= 1911520717;  
    a ^= b << s | b >> w - s;  
    a *= 2048419325;  
  
    float random = a * (3.14159265 / ~(~0u >> 1)); // in [0, 2*Pi]  
  
    // Create the vector from the angle  
    vector2 v;  
    v.x = sin(random);  
    v.z = cos(random);  
    return v;  
}
```

Funzione di hash deterministica

Calcolo del gradiente associato a un vertice della griglia

Rendering Terreno - Perlin Noise

```
float NoiseGenerator::perlin(float x, float z) {  
    int x0 = (int)x;  
    int z0 = (int)z;  
    int x1 = x0 + 1;  
    int z1 = z0 + 1;  
  
    float sx = x - (float)x0;  
    float sz = z - (float)z0;  
  
    float n0 = dotGridGradient(x0, z0, x, z);  
    float n1 = dotGridGradient(x1, z0, x, z);  
    float ix0 = interpolate(n0, n1, sx);  
  
    n0 = dotGridGradient(x0, z1, x, z);  
    n1 = dotGridGradient(x1, z1, x, z);  
    float ix1 = interpolate(n0, n1, sx);  
  
    float value = interpolate(ix0, ix1, sz);  
  
    return value;  
}
```

Prodotto scalare tra gradiente e vettore distanza

Interpolazioni

Calcolo del rumore a frequenza unitaria di un punto (x, z)

Rendering Terreno - Perlin Noise

```
float NoiseGenerator::get_noise(float x, float z) {  
    float temp_freq = freq;  
    float temp_amp = amp;
```

```
    float noise = 0;
```

```
    for (int i = 0; i < n_octaves; i++) {
```

```
        float perlin = NoiseGenerator::perlin(x * temp_freq, z * temp_freq);
```

```
        noise += perlin * temp_amp;
```

```
        temp_freq *= 2;
```

```
        temp_amp /= 2;
```

```
    }
```

```
    noise *= 1.2; // Contrast
```

```
    if (noise > 1.0f) {
```

```
        noise = 1.0f;
```

```
    } else if (noise < -1.0f) {
```

```
        noise = -1.0f;
```

```
    }
```

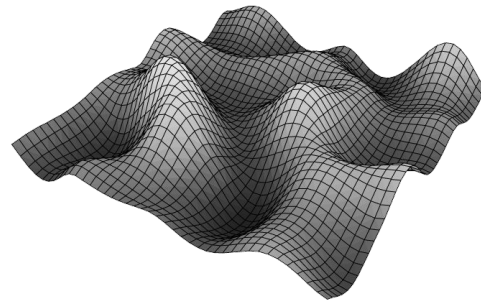
```
    return noise;
```

```
}
```

Somma dei layer di rumore: ogni layer raddoppia la frequenza e dimezza l'ampiezza

Clamping del risultato

Falloff



Rendering Terreno - Falloff

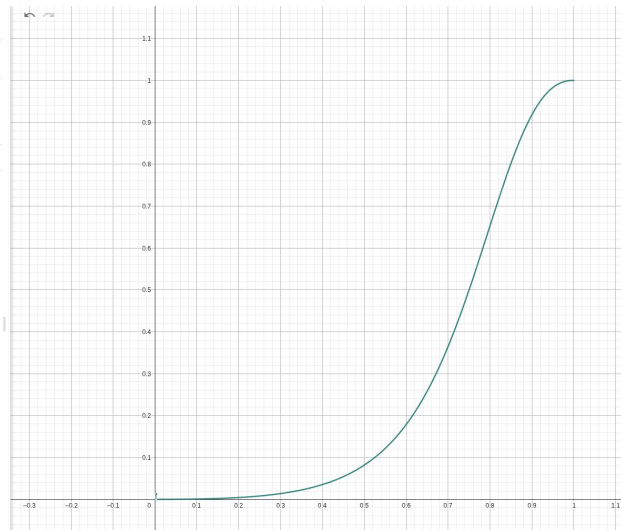
```
float FalloffGenerator::get_vertex_falloff(float x, float z,
                                          float world_width,
                                          float world_height)
{
    float i = x / (float)world_width * 2 - 1;
    float j = z / (float)world_height * 2 - 1;
    }
    float value = glm::max(glm::abs(i), glm::abs(j));
    return evaluate(value);
}

float FalloffGenerator::evaluate(float value)
{
    float a = 3;
    float b = 2.2f;

    return glm::pow(value, a) / (glm::pow(value, a) + glm::pow(b - b * value, a));
}
```

Calcola coordinate compresse in [0, 1]

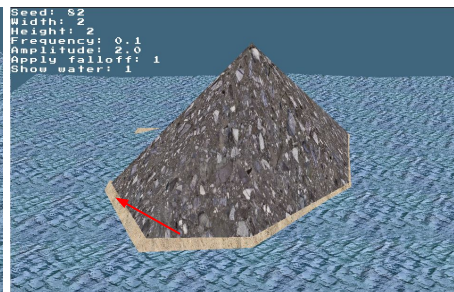
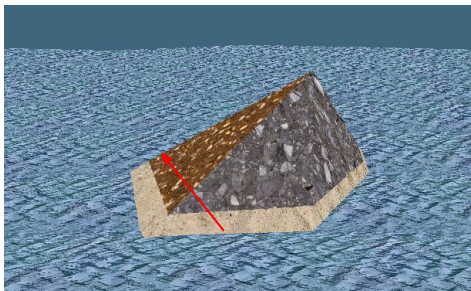
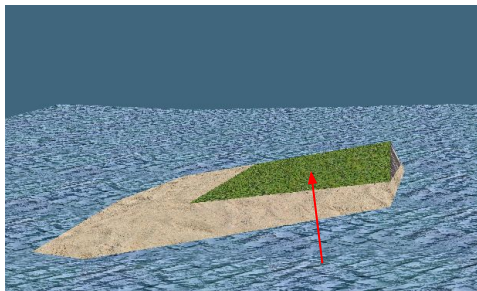
$$f: \frac{x^a}{x^a + (b - bx)^a}$$
$$= Y = \frac{x^{2.2}}{x^{2.2} + (3 - 3x)^{2.2}}$$



Rendering Terreno - Texture

Il *fragment shader* colora ogni superficie:

- $y \leq 0.05 \rightarrow$ spiaggia
- $y > 0.05$: dipende dalla normale n della superficie
 - $n.y \geq 0.9 \rightarrow$ pianura
 - $n.y \geq 0.8 \rightarrow$ montagna
 - Altrimenti $\rightarrow$ roccia



Repository GitHub

<https://github.com/andrearanica/procedural-generation>

Fonti

- C++ Perlin Noise: <https://www.youtube.com/watch?v=kClaHqb60Cw>
- Procedural Generation: https://en.wikipedia.org/wiki/Procedural_generation
- Perlin Noise: https://en.wikipedia.org/wiki/Perlin_noise
- Water generation: <https://www.youtube.com/watch?v=5yhDb9dzJ58>
- Falloff function: https://www.youtube.com/watch?v=COmtTyLCd6I&list=PLFt_AvWsXI0eBW2EiBtl_sxmDtSgZBxB3&index=11
- <https://medium.com/@sashminadhikari/introduction-to-opengl-procedural-terrain-generation-using-c-dd1d981eebd5>